CS-562 Advanced Topics in Databases
Big Data Processing & Analytics
Course Project B: SumQA

Semantic graph summaries as schema context for LLM text-to-SPARQL

Announced: 2 November 2026 · Presentations: 14 and 16 December 2026 · Report: 15 January 2027, 23:59

teams of 5 · 3 teams on this project · submission through eLearn

In one paragraph. An LLM that writes SPARQL over a large knowledge graph has to know which classes and properties exist and how they connect. The full ontology does not fit in the prompt, so current systems pass retrieved schema fragments. Semantic graph summaries (Lecture 8) describe the structure of a whole graph in a few hundred nodes and edges. In this project every team builds the same text-to-SPARQL pipeline with two baselines, then adds one kind of summary and measures which context gives the most correct queries per prompt token. The three summaries together form one comparison, which we plan to write up as a research paper with the students whose results it uses.

Background. Lecture 8 (quotient summaries: weak, strong, typed; pattern and statistical summaries; strong summary in PySpark), Lecture 11 (text-to-SQL, schema linking, RAG over KGs, cost model, batch LLM calls), Lecture 3 (retrieval), Lecture 4 (Spark SQL).

1. The problem

Input: a question in English, the IRIs of the entities it mentions, and a **context** of at most B tokens that describes the schema of the KG. Output: a SPARQL query. The query is run on the KG and its answers are compared with the answers of the gold query. We ask three questions:

1. **RQ1.** For a fixed budget B in {500, 1000, 2000} tokens, which context gives the highest accuracy: the ontology, retrieved schema elements, or a summary?
2. **RQ2.** Does a question-specific slice of a summary keep the accuracy of the whole summary with fewer tokens?
3. **RQ3.** Which parts of the context does the model need? A counterfactual ablation removes blocks of context lines one at a time and records which removals turn a correct answer into a wrong one.

Entity linking is not part of the project: every setting gets the resource IRIs of the gold query, so that the comparison isolates the effect of the schema context.

2. Common part: pipeline and baselines (checkpoint 1, 20%)

1. Set everything up yourselves, following `SETUP.md` of the kit (`06_Projects/Project_B_SumQA/kit`): run the toy pipeline, download DBpedia and dblp, load them into local Oxigraph stores (SPARQL without a server), install Ollama and pull `qwen2.5-coder:7b`, and commit the `SETUP_REPORT.txt` written by `scripts/check_setup.py`.
2. Compute the gold answers (`sumqa.gold`) and check the list of dropped questions.
3. Run the two baselines, **ontology** (schema elements ordered by usage) and **retrieval** (BM25 over schema elements), at budgets 500, 1000 and 2000 on QALD-9-plus test and the LC-QuAD 1.0 sample, with model M1.
4. Score every run (`sumqa.score --results results/results_B.csv`) and fill the error sheet for 30 wrong answers of the retrieval baseline at budget 1000 (`--errors`).

Checkpoint 1 test. Your baseline numbers must match the numbers of the other two teams on the same settings (the runs are deterministic: temperature 0, fixed examples, fixed prompt). Differences point to a setup problem, and we fix them together at the checkpoint.

3. Team variants (checkpoint 2 and final, 30%)

Each team computes its summary with PySpark on the full KG (the Parquet files of the set-up), writes it in the summary format of the kit (`sumqa/context.py`), and runs it through the same pipeline. All three teams work on DBpedia 2016-10 first and then on dblp 2022.

B1. Quotient summaries

- Implement the **weak** and the **strong** summary of Lecture 8 (property cliques; nodes as equivalence classes of resources), with types attached to summary nodes as data-then-type.
- Use the PySpark strong-summary code of Lecture 8 as a starting point; make it run on the full DBpedia files of the course in reasonable time and report the time.
- Report: accuracy per budget for weak and strong; how many summary nodes and edges fit in each budget; which questions a quotient summary answers that the baselines miss.

B2. Type-based and pattern summaries

- Implement a **type-based** summary (a node per class, or per set of classes) and an **ABSTAT-style pattern** summary: patterns (C1, p, C2) with frequencies, keeping the minimal types of subject and object.
- Study two orderings of patterns within the budget: by frequency, and by a coverage criterion of your design (for example, at least one pattern per property before the second pattern of any property).
- Report: accuracy per budget for both summaries and both orderings; the effect of `dbp:` infobox properties (DBpedia) on accuracy and size.

B3. Question-specific slices

- Compute a strong or typed summary once (your own PySpark code), then for every question cut a **slice**: the part of the summary reachable within k hops from the summary nodes of the linked entities, plus the classes and properties that BM25 matches to the question.
- Write the slices with `--context lines`. Grid: k in {1, 2}; with and without the BM25 part.
- Report: accuracy and mean context tokens against the whole summary and the retrieval baseline; RQ2 is your main question.

All teams. Run the ablation (`sumqa.ablation`, 40 questions) for your best setting at budget 1000 and report the necessity rate and the share of necessary tokens (RQ3).

4. Data, models and kit

KG and version	Benchmarks (test questions used)	Notes
DBpedia 2016-10, English core files + ontology	QALD-9-plus DBpedia (150); LC-QuAD 1.0 (300 sampled from 1,000)	LC-QuAD uses many `dbp:` properties; `infobox_properties` is loaded
dblp RDF dump 2022-08-01	DBLP-QuAD (300 sampled from 2,000)	the KG version of DBLP-QuAD
Toy KG (in the kit)	12 questions	development without server or model

Training splits (QALD-9-plus 408, LC-QuAD 4,000, DBLP-QuAD 7,000) are used only for the 3 fixed examples in the prompt. Each team builds, from the same files, a local SPARQL store per KG (`sumqa.kgstore`) and a Parquet copy for its Spark code (`sumqa.kgstats`); since all teams start from the same files, all get the same gold answers. Plan about 50 GB of disk on one machine of the team.

Models. M1 = `qwen2.5-coder:7b` (all runs), M2 = `llama3.1:8b` or `qwen3:8b` (budget 1000 only), through Ollama on the team's own machines. Measure the seconds per question in the set-up smoke test and plan the runs with it; answers are cached, so a run that stops can be resumed at no cost. Commercial APIs are not required and not used in the comparison.

5. Experimental protocol

Runs (minimum)	Model	Benchmarks	Budgets

ontology, retrieval (checkpoint 1)	M1	QALD-9-plus, LC-QuAD	500, 1000, 2000
your variant(s)	M1	QALD-9-plus, LC-QuAD	500, 1000, 2000
ontology, retrieval, best variant	M2	QALD-9-plus, LC-QuAD	1000
ontology, retrieval, best variant	M1	DBLP-QuAD	1000
ablation of the best variant	M1	QALD-9-plus (40 questions)	1000

- Do not change the prompt, the examples or the scorer. Prompt studies are welcome as an extra, clearly separated experiment.
- Every scored run appends one row to the results table; keep the run files (they are cached and small).
- Report the time to compute your summary in Spark, its size (nodes, edges, tokens) and the machine used.
- Plots: (a) accuracy against mean context tokens for all settings (one point per setting and budget); (b) outcome counts (correct, partial, wrong, empty, error) per setting at budget 1000.

6. Milestones

Date	Milestone	What we check
Mon 2 Nov	Kick-off (Project Discussion)	teams and variants fixed
Mon 9 Nov	Set-up done	`SETUP_REPORT.txt` committed, all checks OK (stores, gold answers, Ollama)
Mon 16 Nov	Checkpoint 1 (15 min per team)	baseline rows in the results table, equal across teams
Mon 30 Nov	Checkpoint 2 (15 min per team)	summary computed on DBpedia; first runs at budget 1000
14 or 16 Dec	Presentation (15 min + 5 min questions)	all minimum runs; first ablation numbers
Fri 15 Jan 2027	Report and code	6-page report, repository tag `final`

7. Deliverables and grading

Part	Weight	Content
Set-up, pipeline and baselines (checkpoint 1)	20%	set-up report on time; baseline runs in the results table; error sheet of 30 questions
Variant	30%	summary code in PySpark, runs of section 5, ablation
Report	30%	6 pages, LNCS format: problem, summary design, experiments, error analysis, discussion
Presentation	20%	15 minutes, all members present; one question followed from summary to query to answer is required

Each team is graded on its own pipeline and variant. A summary that does not beat the baselines but is implemented, measured and explained honestly can still get full marks: negative results are results.

8. Rules

- One repository per team (GitHub or the CSD GitLab), shared with the instructor from day 1. Commit regularly; the history is part of the grade of each member.
- Teams of this project may help each other with the set-up, the kit and the models. Code is written per team.
- Use of AI assistants follows the course policy (Lecture 1): allowed, declared in the report, and every member must be able to explain every part of the code.
- The results may be used in a research paper by the instructor. Students whose work is used will be invited as co-authors.

9. Reading

- Š. Čebirić, F. Goasdoué, H. Kondylakis, D. Kotzinos, I. Manolescu, G. Troullinou, M. Zneika. Summarizing semantic graphs: a survey. The VLDB Journal 28(3), 2019.
- F. Goasdoué, P. Guzewicz, I. Manolescu. RDF graph summarization for first-sight structure discovery. The VLDB Journal 29(5), 2020.
- B. Spahiu et al. Understanding the structure of knowledge graphs with ABSTAT profiles. Semantic Web 15(5), 2024.
- L. Wardenga, T. Käfer. Leveraging Data Shapes in LLM Contexts for QA on Public and Private KGs. TEXT2SPARQL Challenge, ESWC 2025.
- J. D'Abramo, A. Zugarini, P. Torroni. Investigating Large Language Models for Text-to-SPARQL Generation. KnowledgeNLP workshop, 2025.
- L. Kovriguina et al. SPARQLGEN: One-Shot Prompt-based Approach for SPARQL Query Generation. SEMANTiCS 2023.
- D. Allemang, J. Sequeda. Increasing the LLM Accuracy for Question Answering: Ontologies to the Rescue! arXiv:2405.11706, 2024.
- A. Perevalov, D. Diefenbach, R. Usbeck, A. Both. QALD-9-plus. ICSC 2022.
- P. Trivedi, G. Maheshwari, M. Dubey, J. Lehmann. LC-QuAD: A Corpus for Complex Question Answering over Knowledge Graphs. ISWC 2017.
- D. Banerjee, S. Awale, R. Usbeck, C. Biemann. DBLP-QuAD: A Question Answering Dataset over the DBLP Scholarly Knowledge Graph. arXiv:2303.13351, 2023.